

杂题选讲

Nesraychan

Tsinghua University

July 16, 2026

[2022-2023 集训队互测] 树

有一棵包含 n 个节点的隐藏树（边权均为 1），要求通过有限次查询确定整棵树的所有边。

每次可以询问一个点 u 到一个点集 V 的距离之和。

满足 $n \leq 1000$ ，且总查询次数不能超过 8500 次，所查询集合大小的总和不能超过 3×10^5 。

交互库不自适应。

[2022-2023 集训队互测] 树

第二个限制比较松，我们主要考虑第一个限制。随机一个根然后分层，问题在于第 i 层的点 A 到第 $i+1$ 层的点 B 之间的边。

[2022-2023 集训队互测] 树

第二个限制比较松，我们主要考虑第一个限制。随机一个根然后分层，问题在于第 i 层的点 A 到第 $i+1$ 层的点 B 之间的边。

考虑分治，每次在 A 中随机一个集合 S 。然后我们可以通过已知的前 i 层和询问求出 A, B 中每个点到 S 的距离。如果是父子关系肯定有差为 $|S|$ ，那么就可以求出等价类递归下去。

[2022-2023 集训队互测] 树

第二个限制比较松，我们主要考虑第一个限制。随机一个根然后分层，问题在于第 i 层的点 A 到第 $i+1$ 层的点 B 之间的边。

考虑分治，每次在 A 中随机一个集合 S 。然后我们可以通过已知的前 i 层和询问求出 A, B 中每个点到 S 的距离。如果是父子关系肯定有差为 $|S|$ ，那么就可以求出等价类递归下去。

考虑在最坏情况下，期望意义下等价类至少折半。所以期望询问数是不超过 $n \log n$ 的，大小和是 n^2 。但由于是随机，且这个上界是肯定不满的，所以可以轻松通过。

[ABC238 Ex] Removing People

有 n 人站成一个圆圈，按顺时针方向依次为 $1, 2, \dots, n$ 。

每个人面朝的方向由长度为 n 的字符串 s 给出。对于第 i 个人，如果 $s_i = L$ ，则 i 面向逆时针方向；如果 $s_i = R$ ，则面向顺时针方向。

重复 $n-1$ 次如下操作：以相等的概率从剩余的人中选择一个，并移除其面朝方向的第一个人，代价等于被选中的人到被移除的人的距离。

求 $n-1$ 次操作后代价之和的期望值，对 998244353 取模。

$1 \leq n \leq 300$.

[ABC238 Ex] Removing People

正难则反，我们考虑倒过来。那么一次操作变成选择一个点，从该方向下一个人之前选择一个人加入。

[ABC238 Ex] Removing People

正难则反，我们考虑倒过来。那么一次操作变成选择一个点，从该方向下一个人之前选择一个人加入。

那么你会发现这样加人不会超出相邻两个人的区间，所以我们考虑直接区间 dp，设 $f(l, r)$ 表示 $[l, r]$ 的方案数。

[ABC238 Ex] Removing People

正难则反，我们考虑倒过来。那么一次操作变成选择一个点，从该方向下一个人之前选择一个人加入。

那么你会发现这样加人不会超出相邻两个人的区间，所以我们考虑直接区间 dp，设 $f(l, r)$ 表示 $[l, r]$ 的方案数。

转移时只需要枚举区间内下一个出现的人是谁即可，两侧区间独立，方案数就是归并。

[ABC238 Ex] Removing People

正难则反，我们考虑倒过来。那么一次操作变成选择一个点，从该方向下一个人之前选择一个人加入。

那么你会发现这样加人不会超出相邻两个人的区间，所以我们考虑直接区间 dp，设 $f(l, r)$ 表示 $[l, r]$ 的方案数。

转移时只需要枚举区间内下一个出现的人是谁即可，两侧区间独立，方案数就是归并。

时间复杂度 $O(n^3)$ 。

[CTT2024] 前往何方

有一棵包含 n 个节点的隐藏树，你需要维护一个初始为空的点集 S 。

每次操作可以向 S 中增加或删除一个节点，然后交互库会返回当前 S 导出子图的最大连通块大小。

要求通过这些信息还原整棵树的所有边。

交互库不自适应。

$1 \leq n \leq 10^4$ ，增加和删除操作的总调用次数不得超过 6×10^5 次。

[CTT2024] 前往何方

注意到我们按一个随机顺序贪心求一个独立集，这个独立集的期望大小是 $\sum \frac{1}{d_i+1}$ ，所以在最劣情况下，这个期望仍然是 $\frac{1}{3}n$ 。

[CTT2024] 前往何方

注意到我们按一个随机顺序贪心求一个独立集，这个独立集的期望大小是 $\sum \frac{1}{d_i+1}$ ，所以在最劣情况下，这个期望仍然是 $\frac{1}{3}n$ 。

所以我们先求出这样一个独立集 S ，同时可以方便知道其他点有多少连向独立集的边。

[CTT2024] 前往何方

注意到我们按一个随机顺序贪心求一个独立集，这个独立集的期望大小是 $\sum \frac{1}{d_i+1}$ ，所以在最劣情况下，这个期望仍然是 $\frac{1}{3}n$ 。

所以我们先求出这样一个独立集 S ，同时可以方便知道其他点有多少连向独立集的边。

那么一个点 x 到 S 的第 k 条边，事实上就是 S 最小的一个前缀，使得 $S \cup \{x\}$ 有大小为 $k+1$ 的连通块，那么可以直接整体二分。

[CTT2024] 前往何方

注意到我们按一个随机顺序贪心求一个独立集，这个独立集的期望大小是 $\sum \frac{1}{d_i+1}$ ，所以在最劣情况下，这个期望仍然是 $\frac{1}{3}n$ 。

所以我们先求出这样一个独立集 S ，同时可以方便知道其他点有多少连向独立集的边。

那么一个点 x 到 S 的第 k 条边，事实上就是 S 最小的一个前缀，使得 $S \cup \{x\}$ 有大小为 $k+1$ 的连通块，那么可以直接整体二分。

操作复杂度 $O(n \log n)$ ，常数很小，随便过。本题存在不依赖于非自适应的点分治做法，但需要仔细实现。场上尝试写点分治的人基本没有卡进限制里的。

[CF2096G]Wonderful Guessing Game

Alice 心中有一个 1 到 n 之间的隐藏整数，要求通过非适应性查询（即必须一次性提出所有查询）确定这个数。

每次可以给出一个由 k 个不同整数组成的偶数长度数组。交互库会回应目标数是在数组前半部分、后半部分、还是不在数组中。但在最终的所有回复中，会有恰好一个查询被忽略。

满足 $2 \leq n \leq 2 \times 10^5$ ，总查询次数 q 必须等于确定答案所需的理论最小次数 $f(n)$ （且保证 $f(n) \leq 20$ ）。

交互库自适应。

[CF2096G]Wonderful Guessing Game

首先交互库肯定可以一直不告诉你第一个询问，所以必须有
 $3^{q-1} \geq n$ 。

[CF2096G]Wonderful Guessing Game

首先交互库肯定可以一直不告诉你第一个询问，所以必须有 $3^{q-1} \geq n$ 。

然后我们证明一定可以取到下界，考虑这是事实就是给所有人一个三进制下的编号，要求任意两个人至少有两个不同位置。于是对于一个数 x ，我们把前 $q-1$ 个位置设为其三进制下的表示，那么现在两个数有至少一个位置不同。

[CF2096G]Wonderful Guessing Game

首先交互库肯定可以一直不告诉你第一个询问，所以必须有 $3^{q-1} \geq n$ 。

然后我们证明一定可以取到下界，考虑这是事实就是给所有人一个三进制下的编号，要求任意两个人至少有两个不同位置。于是对于一个数 x ，我们把前 $q-1$ 个位置设为其三进制下的表示，那么现在两个数有至少一个位置不同。

我们将第 q 为设为前 $q-1$ 位模三下的和，这样如果前面只有一个位置不同，这个位置肯定不同。

[CF2096G]Wonderful Guessing Game

首先交互库肯定可以一直不告诉你第一个询问，所以必须有 $3^{q-1} \geq n$ 。

然后我们证明一定可以取到下界，考虑这是事实就是给所有人一个三进制下的编号，要求任意两个人至少有两个不同位置。于是对于一个数 x ，我们把前 $q-1$ 个位置设为其三进制下的表示，那么现在两个数有至少一个位置不同。

我们将第 q 为设为前 $q-1$ 位模三下的和，这样如果前面只有一个位置不同，这个位置肯定不同。

于是这就符合要求了。但这个题还有一个额外限制是每一位是 0 和是 1 的要一样多，不过这就是细节问题了，这里不过多进行赘述。

[AGC067B] Modification

有一个长度为 n 的序列 $a_1, a_2, \dots, a_n$, 初始均为 0 ; 以及 m 个可行的覆盖区间 $[l_1, r_1], [l_2, r_2], \dots, [l_m, r_m]$ 。

接下来你每次会选择一个可行的覆盖区间 $[l_i, r_i]$ 和一个 $[1, k]$ 内的整数 c , 并将 $a_{l_i}, a_{l_i+1}, \dots, a_{r_i}$ 修改为 c 。

你需要选择每个区间恰好一次, 问有多少种最终可能的 a 。
答案对 998244353 取模。

$$1 \leq n \leq 100, 1 \leq m \leq \frac{n(n+1)}{2}, 1 \leq k < 998244353.$$

[AGC067B] Modification

首先考虑怎么 check，发现我们可以按照时间从后往前能删就删。

[AGC067B] Modification

首先考虑怎么 check，发现我们可以按照时间从后往前能删就删。

基于这个 check，发现计算什么东西能被删完是困难的，但计算不能被删完并不是如此。

[AGC067B] Modification

首先考虑怎么 check，发现我们可以按照时间从后往前能删就删。

基于这个 check，发现计算什么东西能被删完是困难的，但计算不能被删完并不是如此。

具体来说，对于所有未被删除的位置，要求所有仍然有交的区间在上面的颜色数量至少为 2。这是一个十分经典的问题。

[AGC067B] Modification

首先考虑怎么 check，发现我们可以按照时间从后往前能删就删。

基于这个 check，发现计算什么东西能被删完是困难的，但计算不能被删完并不是如此。

具体来说，对于所有未被删除的位置，要求所有仍然有交的区间在上面的颜色数量至少为 2。这是一个十分经典的问题。

而被删除的位置会形成若干极长段，段内互相独立。

[AGC067B] Modification

首先考虑怎么 check，发现我们可以按照时间从后往前能删就删。

基于这个 check，发现计算什么东西能被删完是困难的，但计算不能被删完并不是如此。

具体来说，对于所有未被删除的位置，要求所有仍然有交的区间在上面的颜色数量至少为 2。这是一个十分经典的问题。

而被删除的位置会形成若干极长段，段内互相独立。

于是可以简单 dp，时间复杂度 $O(n^4)$ 。

[IOI2025 Practice] Magic Trick

有一个包含 N 个元素的隐藏排列 P ，要求通过 Alicia 和 Beatriz 两个阶段的代码协作，在隐藏部分卡牌的情况下准确还原出原排列。

第一阶段 Alicia 观察完整排列，选择 K 张牌翻面隐藏；

第二阶段 Beatriz 根据留下的卡牌配置，必须确定所有被隐藏卡牌的数值并输出原排列。

满足 $N = 256$ ，所有元素互不相同且在 1 到 N 之间。你需要保证 $K \geq 16$ 。

两阶段程序独立运行，各调用一次。

[IOI2025 Practice] Magic Trick

我们自然希望所有删去的东西是递增或者递减的。

[IOI2025 Practice] Magic Trick

我们自然希望所有删去的东西是递增或者递减的。

由 Dilworth 定理, LIS 的长度乘 LDS 的长度 ≥ 256 , 所以至少有一个 ≥ 16 。

[IOI2025 Practice] Magic Trick

我们自然希望所有删去的东西是递增或者递减的。

由 Dilworth 定理, LIS 的长度乘 LDS 的长度 ≥ 256 , 所以至少有一个 ≥ 16 。

那么若 LIS 长度 ≥ 16 , 我们从中选一个长度为 16 的上升子序列删掉。

[IOI2025 Practice] Magic Trick

我们自然希望所有删去的东西是递增或者递减的。

由 Dilworth 定理, LIS 的长度乘 LDS 的长度 ≥ 256 , 所以至少有一个 ≥ 16 。

那么若 LIS 长度 ≥ 16 , 我们从中选一个长度为 16 的上升子序列删掉。

反之 LDS 长度 ≥ 17 , 我们从中选一个长度为 17 的下降子序列删掉。这样还原的时候只需要通过删了多少数来判断是哪一种。

[1st UCUP Ukraine Stage]Most Annoying Constructive Problem

给定正整数 n 和 k , 要求构造一个 1 到 n 的排列, 使得该排列的所有连续子数组中, 逆序对数量为奇数的子数组个数恰好为 k 。

判断是否有解, 如果有解, 构造一个这样的排列。

满足 $1 \leq n \leq 1000, 0 \leq k \leq \frac{n(n-1)}{2}$, 且所有测试点的 $\sum n^2 \leq 4 \times 10^6$

[1st UCUP Ukraine Stage]Most Annoying Constructive Problem

首先我们只考虑所有长度为 > 1 的子区间。存在定理：对于 $n \geq 4$ ，至少存在 $\lfloor \frac{n-1}{2} \rfloor$ 个偶数子区间。

[1st UCUP Ukraine Stage] Most Annoying Constructive Problem

首先我们只考虑所有长度为 > 1 的子区间。存在定理：对于 $n \geq 4$ ，至少存在 $\lfloor \frac{n-1}{2} \rfloor$ 个偶数子区间。

对于 $n = 4, 5$ ，可以暴力验证。接着考虑归纳，如果 n 偶数子区间的数量比 $n - 2$ 没有增加，说明 $p_{n-2} > p_{n-1} > p_n$ 。

[1st UCUP Ukraine Stage] Most Annoying Constructive Problem

首先我们只考虑所有长度为 > 1 的子区间。存在定理：对于 $n \geq 4$ ，至少存在 $\lfloor \frac{n-1}{2} \rfloor$ 个偶数子区间。

对于 $n = 4, 5$ ，可以暴力验证。接着考虑归纳，如果 n 偶数子区间的数量比 $n - 2$ 没有增加，说明 $p_{n-2} > p_{n-1} > p_n$ 。

接着可以知道任何后缀比 p_n 大的数量一定是偶数，所以可知 $[1, n - 3]$ 都要比 p_n 小，这样就矛盾了。

[1st UCUP Ukraine Stage] Most Annoying Constructive Problem

首先我们只考虑所有长度为 > 1 的子区间。存在定理：对于 $n \geq 4$ ，至少存在 $\lfloor \frac{n-1}{2} \rfloor$ 个偶数子区间。

对于 $n = 4, 5$ ，可以暴力验证。接着考虑归纳，如果 n 偶数子区间的数量比 $n - 2$ 没有增加，说明 $p_{n-2} > p_{n-1} > p_n$ 。

接着可以知道任何后缀比 p_n 大的数量一定是偶数，所以可知 $[1, n - 3]$ 都要比 p_n 小，这样就矛盾了。

接着考虑给所有不超过这个界的情况一个构造。

[1st UCUP Ukraine Stage] Most Annoying Constructive Problem

我们考虑归纳构造, 设 $f(n)$ 表示 n 对应的最大可行 k : 若 n 较小, 可以直接暴力。

[1st UCUP Ukraine Stage] Most Annoying Constructive Problem

我们考虑归纳构造, 设 $f(n)$ 表示 n 对应的最大可行 k : 若 n 较小, 可以直接暴力。

若 $k = 0$, 直接 $1, 2, \dots, n$ 即可;

[1st UCUP Ukraine Stage] Most Annoying Constructive Problem

我们考虑归纳构造, 设 $f(n)$ 表示 n 对应的最大可行 k : 若 n 较小, 可以直接暴力。

若 $k = 0$, 直接 $1, 2, \dots, n$ 即可;

若 $0 < k \leq n - 2$, 考虑 $1, 2, \dots, k + 2, k, k + 1, k + 3, \dots, n$;

[1st UCUP Ukraine Stage] Most Annoying Constructive Problem

我们考虑归纳构造, 设 $f(n)$ 表示 n 对应的最大可行 k : 若 n 较小, 可以直接暴力。

若 $k = 0$, 直接 $1, 2, \dots, n$ 即可;

若 $0 < k \leq n - 2$, 考虑 $1, 2, \dots, k + 2, k, k + 1, k + 3, \dots, n$;

若 $n - 1 \leq k \leq f(n - 2) + n - 1$, 在最后两个位置放 $(n, n - 1)$, 则可以递归到 $(n - 2, k - n + 1)$;

[1st UCUP Ukraine Stage] Most Annoying Constructive Problem

我们考虑归纳构造, 设 $f(n)$ 表示 n 对应的最大可行 k : 若 n 较小, 可以直接暴力。

若 $k = 0$, 直接 $1, 2, \dots, n$ 即可;

若 $0 < k \leq n - 2$, 考虑 $1, 2, \dots, k + 2, k, k + 1, k + 3, \dots, n$;

若 $n - 1 \leq k \leq f(n - 2) + n - 1$, 在最后两个位置放 $(n, n - 1)$, 则可以递归到 $(n - 2, k - n + 1)$;

若 $k = f(n)$, 考虑这样一个序列: $4, 1, 6, 3, 8, 5, 10, 7, \dots$, 可以发现所有偶数子区间只有 $[2i, 2i + 1]$ 。于是将这个序列离散化就是一组解。

[1st UCUP Ukraine Stage]Most Annoying Constructive Problem

若 $f(n-2) + n \leq k < f(n)$, 这时存在这样一个结论: 一定存在一组解, 是由 $n-2$ 的最大解, 首尾各插入一个数得到。

[1st UCUP Ukraine Stage]Most Annoying Constructive Problem

若 $f(n-2) + n \leq k < f(n)$, 这时存在这样一个结论: 一定存在一组解, 是由 $n-2$ 的最大解, 首尾各插入一个数得到。

感性思考一下这个为什么是对的: 这相当于你要新增 $[n, 2n-5]$ 的区间, 然后首尾是独立的, 相当于你有 $O(n^2)$ 个方案去覆盖一个长度为 $O(n)$ 的区间。

[1st UCUP Ukraine Stage]Most Annoying Constructive Problem

若 $f(n-2) + n \leq k < f(n)$, 这时存在这样一个结论: 一定存在一组解, 是由 $n-2$ 的最大解, 首尾各插入一个数得到。

感性思考一下这个为什么是对的: 这相当于你要新增 $[n, 2n-5]$ 的区间, 然后首尾是独立的, 相当于你有 $O(n^2)$ 个方案去覆盖一个长度为 $O(n)$ 的区间。

在题目范围内, 可以写代码验证这都是对的。同时这是存在组合证明的方法的, 可惜这里地方太小, 写不下。于是给大家留作练习 (

[1st UCUP Ukraine Stage] Most Annoying Constructive Problem

若 $f(n-2) + n \leq k < f(n)$, 这时存在这样一个结论: 一定存在一组解, 是由 $n-2$ 的最大解, 首尾各插入一个数得到。

感性思考一下这个为什么是对的: 这相当于你要新增 $[n, 2n-5]$ 的区间, 然后首尾是独立的, 相当于你有 $O(n^2)$ 个方案去覆盖一个长度为 $O(n)$ 的区间。

在题目范围内, 可以写代码验证这都是对的。同时这是存在组合证明的方法的, 可惜这里地方太小, 写不下。于是给大家留作练习 (

时间复杂度可以通过预处理做到 $O(n^2)$ 。

[CF1746E]Joking

有 n 个元素，其中有一个是特别的，你需要找到它。

你每次可以问一个集合，会得到元素是否在集合中。

但是交互库可能说谎，我们只保证交互库不会连续说谎两次。

由于在上述条件中分辨 $n = 2$ 是不可能的，你也可以回答两次。

$n \leq 10^5$ ，有 $82/53$ 次询问机会。

[CF1746E1]Joking(Easy Version)

如何快速缩小范围？尝试去掉 $O(n)$ 个数。

[CF1746E1]Joking(Easy Version)

如何快速缩小范围？尝试去掉 $O(n)$ 个数。

假设有四个数 A, B, C, D ，我们询问 $\{A, B\}$ 和 $\{A, C\}$ 之后一定可以排除一个元素。将数换为集合也成立，相当于花费两次询问去掉 $\lfloor n/4 \rfloor$ 个数。

[CF1746E1]Joking(Easy Version)

如何快速缩小范围？尝试去掉 $O(n)$ 个数。

假设有四个数 A, B, C, D ，我们询问 $\{A, B\}$ 和 $\{A, C\}$ 之后一定可以排除一个元素。将数换为集合也成立，相当于花费两次询问去掉 $\lfloor n/4 \rfloor$ 个数。

82 次询问非常够用，最后只需要处理 $n = 3$ 的情况：

这时我们先问 $\{1, 2\}$ 。如果是 NO 我们回答 3 就可以知道刚刚是不是说谎；否则再问一次 $\{1, 2\}$ ，要么可以转换成上面，要么可以直接确定。

[CF1746E2]Joking(Hard Version)

Hard Version 需要我们更精细地选择集合，我们刚刚的问题是将两个询问划成一组，不同组之间最多说谎两次的限制根本没有用上。

[CF1746E2]Joking(Hard Version)

Hard Version 需要我们更精细地选择集合，我们刚刚的问题是将两个询问划成一组，不同组之间最多说谎两次的限制根本没有用上。

考虑一般化这个问题，一个时刻我们知道答案肯定在一个集合 S 里，而这个集合的一个子集 $T \subseteq S$ 是上次交互库告诉我们这个数在的集合。

[CF1746E]Joking(Hard Version)

Hard Version 需要我们更精细地选择集合，我们刚刚的问题是将两个询问划成一组，不同组之间最多说谎两次的限制根本没有用上。

考虑一般化这个问题，一个时刻我们知道答案肯定在一个集合 S 里，而这个集合的一个子集 $T \subseteq S$ 是上次交互库告诉我们这个数在的集合。

我们只关心 $|S|, |T|$ ，所以我们存在一个 $O(n^4)$ 的 dp 求出最优划分。但这是显然过不去的。

[CF1746E]Joking(Hard Version)

Hard Version 需要我们更精细地选择集合，我们刚刚的问题是将两个询问划成一组，不同组之间最多说谎两次的限制根本没有用上。

考虑一般化这个问题，一个时刻我们知道答案肯定在一个集合 S 里，而这个集合的一个子集 $T \subseteq S$ 是上次交互库告诉我们这个数在的集合。

我们只关心 $|S|, |T|$ ，所以我们存在一个 $O(n^4)$ 的 dp 求出最优划分。但这是显然过不去的。

不过在较大的情况肯定还是像 Easy Version 那么划分的，那么我们设一个阈值就可以做到 $O(B^4)$ 。这里设 $B = 20$ 应该就可以轻松的通过 53 次的限制。

[CF1951F]Inversion Composition

给定排列 p 和参数 k 。

记 $\text{inv}(a)$ 为排列 a 的逆序对数。

构造排列 q 使得 $\text{inv}(q) + \text{inv}(q \circ p) = k$, 或者报告不可能。

$$(q \circ p)_i = q_{p_i}.$$

$$n \leq 3 \times 10^5.$$

[CF1951F] Inversion Composition

考虑逆序对的定义，我们建两排点，从下面的 i 向上面的 p_i 连边，交点的个数。

[CF1951F] Inversion Composition

考虑逆序对的定义，我们建两排点，从下面的 i 向上面的 p_i 连边，交点的个数。

于是对于这个题，我们建三排点，那么 $q \circ p$ 的逆序对数就是交点数量为奇数的路径。进而我们发现如果在下面两排已经交叉，无论如何都会恰好有一的贡献。

[CF1951F] Inversion Composition

考虑逆序对的定义，我们建两排点，从下面的 i 向上面的 p_i 连边，交点的个数。

于是对于这个题，我们建三排点，那么 $q \circ p$ 的逆序对数就是交点数量为奇数的路径。进而我们发现如果在下面两排已经交叉，无论如何都会恰好有一的贡献。

现在的问题就变成了， q 上的一些数对会产生逆序对贡献，构造一个这类贡献恰好为 w 的 q 。

[CF1951F] Inversion Composition

考虑逆序对的定义，我们建两排点，从下面的 i 向上面的 p_i 连边，交点的个数。

于是对于这个题，我们建三排点，那么 $q \circ p$ 的逆序对数就是交点数量为奇数的路径。进而我们发现如果在下面两排已经交叉，无论如何都会恰好有一的贡献。

现在的问题就变成了， q 上的一些数对会产生逆序对贡献，构造一个这类贡献恰好为 w 的 q 。

判掉奇偶性，然后可以直接贪心构造。时间复杂度 $O(n \log n)$ 。

[AGC049E]Increment Decrement

对于一个初始序列 $a_1, a_2, \dots, a_n$, 你可以以 1 的代价对任一单点 $+1/-1$, 或以 c 的代价对任一区间 $+1, -1$, 你的目标是让所有数变成 0。

现在给定 $S_1, S_2, \dots, S_n$ 。对于初始序列, 其元素 a_i 可以从 S_i 中任意选取。问所有初始序列的选取下达成目标的最小代价之和。答案对 $10^9 + 7$ 取模。

$1 \leq c \leq n \leq 50, 1 \leq |S_i| \leq 50$ 。 S_i 的所有元素都在 $[1, 10^9]$ 内。

[AGC049E]Increment Decrement

考虑给定序列怎么做，一个想法是差分。

[AGC049E]Increment Decrement

考虑给定序列怎么做，一个想法是差分。

那么问题就变成了，我们可以选两个位置 i, j ，一个加一，一个减一，代价为 $\min(c, |i - j|)$ 。显然可以用调整法证明，匹配一定形如括号匹配最优。

[AGC049E]Increment Decrement

考虑给定序列怎么做，一个想法是差分。

那么问题就变成了，我们可以选两个位置 i, j ，一个加一，一个减一，代价为 $\min(c, |i - j|)$ 。显然可以用调整法证明，匹配一定形如括号匹配最优。

但你发现这样做就没救了！现在状态数仍然记不下，因为差分可能很大。不过你发现减法一次只会减一，所以这个过程是一层一层的。我们把同一层的一起做，这样就可以分出 $O(n^2)$ 个本质不同的层。

[AGC049E]Increment Decrement

考虑给定序列怎么做，一个想法是差分。

那么问题就变成了，我们可以选两个位置 i, j ，一个加一，一个减一，代价为 $\min(c, |i - j|)$ 。显然可以用调整法证明，匹配一定形如括号匹配最优。

但你发现这样做就没救了！现在状态数仍然记不下，因为差分可能很大。不过你发现减法一次只会减一，所以这个过程是一层一层的。我们把同一层的一起做，这样就可以分出 $O(n^2)$ 个本质不同的层。

你注意到这样的话，一个前缀只有两种状态：使用第一种消除完了；和使用第二种延伸。这就是一个可以直接 dp of dp 的问题了。于是总的复杂度是 $O(n^5)$ 。

[CF1764G] Doremy's Perfect DS Class

这是一道交互题。

有一个长度为 n 的排列 p 。

你可以询问 q 次 $Q(l, r, k)$ ，交互库会返回 $[\lfloor p_l/k \rfloor, \dots, \lfloor p_r/k \rfloor]$ 中出现的数的种类数。

你需要在 30/25/20 次询问中求出 1 的位置，即求出 $p_x = 1$ 的 x 。

$$3 \leq n \leq 2^{10}。$$

[CF1764G] Doremy's Perfect DS Class

考虑二分，即判定 $x \in [1, r]$ 。

[CF1764G] Doremy's Perfect DS Class

考虑二分，即判定 $x \in [1, r]$ 。

考虑 $k = 2$ ，这时一或两个数一组，一次查询可以改写为查出有多少组恰有一个元素在区间里。

[CF1764G] Doremy's Perfect DS Class

考虑二分，即判定 $x \in [1, r]$ 。

考虑 $k = 2$ ，这时一或两个数一组，一次查询可以改写为查出有多少组恰有一个元素在区间里。

因此如果 n 是奇数，我们已经做完了：1 一定在更多的一侧。

[CF1764G] Doremy's Perfect DS Class

考虑二分，即判定 $x \in [1, r]$ 。

考虑 $k = 2$ ，这时一或两个数一组，一次查询可以改写为查出有多少组恰有一个元素在区间里。

因此如果 n 是奇数，我们已经做完了：1 一定在更多的一侧。

对于偶数的情况，我们发现 1 和 n 都只对一侧产生贡献，我们先暴力用 $\log n$ 次找到 n 的位置再做就可以做到 $3 \log n$ 。

[CF1764G] Doremy's Perfect DS Class

考虑二分，即判定 $x \in [1, r]$ 。

考虑 $k = 2$ ，这时一或两个数一组，一次查询可以改写为查出有多少组恰有一个元素在区间里。

因此如果 n 是奇数，我们已经做完了：1 一定在更多的一侧。

对于偶数的情况，我们发现 1 和 n 都只对一侧产生贡献，我们先暴力用 $\log n$ 次找到 n 的位置再做就可以做到 $3 \log n$ 。

稍微改进一点的方法是我们在二分到两侧分裂数数量相同之后确定 n 在哪一侧，只需要 $2 \log n + 1$ 次。

[CF1764G] Doremy's Perfect DS Class

对于 G3, 我们在分治到一个区间 $[l, r]$ 的时候, 已经知道了 $[1, l-1], [l, n], [r+1, n], [1, r]$ 的信息。

[CF1764G] Doremy's Perfect DS Class

对于 G3, 我们在分治到一个区间 $[l, r]$ 的时候, 已经知道了 $[1, l-1], [l, n], [r+1, n], [1, r]$ 的信息。

考虑最后一步长度为 2 的区间 $[l, l+1]$, 我们可以比较 $[1, l-1], [1, r]$ 的信息, 得知 $[l+1, l]$ 中是否有 n , 或是另一个出现的数在前面是否出现过同组数。

[CF1764G] Doremy's Perfect DS Class

对于 G3, 我们在分治到一个区间 $[l, r]$ 的时候, 已经知道了 $[1, l-1], [l, n], [r+1, n], [1, r]$ 的信息。

考虑最后一步长度为 2 的区间 $[l, l+1]$, 我们可以比较 $[1, l-1], [1, r]$ 的信息, 得知 $[l+1, l]$ 中是否有 n , 或是另一个出现的数在前面是否出现过同组数。

于是我们再查询 $[1, l]$ 即可, 这可以节省一次变成 $2 \log n$ 。